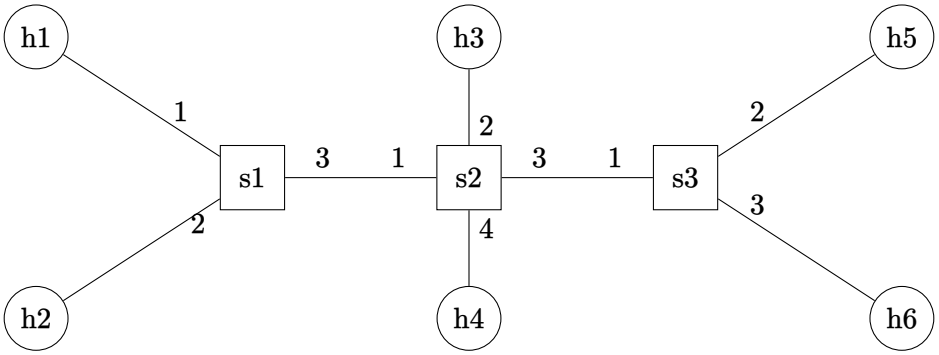
Software Defined Networking
24 giugno 2024

Esercizio 1

Si consideri la rete in figura. I nodi h1,..., hn contengono tabelle ARP prepopolate per tutta la rete. Gli indirizzi MAC e IP dei nodi h1,..., hn sono MAC1,..., MACn e IP1,..., IPn. Sui nodi h1,..., hn sono in esecuzione processi in ascolto su tutte le porte TCP e UDP di interesse. Il controllore Ryu è collegato ai nodi s1,..., sm. La funzione di gestione dell'evento PacketIn è data nel Programma 1. Le interfacce tra gli switch e il controllore non sono indicate in figura. Se non altrimenti specificato, il TTL iniziale è 200.



Negli switch sono preinstallate le regole in Tabella 1.

L'host h1 porta 3322 invia un datagramma UDP verso h3 porta 123.

1. Scrivere se $s1$ invia messaggi al controllore. In caso affermativo scrivere quale tipo di messaggio openflow arriva in risposta dal controllore.

   Soluzione: Sì, riceve una packet-out.

2. Scrivere se $s2$ invia messaggi al controllore. In caso affermativo scrivere quale tipo di messaggio openflow arriva in risposta dal controllore.

   Soluzione: Sì, riceve una flow-mod.

3. Scrivere quali host dell'intera rete ricevono il pacchetto in uscita da $s2$. Per ogni host scrivere il numero di porta UDP destinazione e il TTL.

   Soluzione: Nessun host riceve pacchetti.

4. L'host h1 ritrasmette il pacchetto precedente. Scrivere quali host dell'intera rete ricevono il pacchetto in uscita da $s2$. Per ogni host scrivere il numero di porta UDP destinazione e il TTL.

Soluzione: h5 riceve il pacchetto con porta UDP 123 e TTL 199.

5. L'host che riceve il pacchetto precedente risponde. L'host $h1$ riceve il pacchetto di risposta? In caso affermativo, dire se il pacchetto viene consegnato all'applicazione oppure in quale strato viene scartato.

Soluzione: Il pacchetto viene consegnato all'applicazione.

6. Quale regola si può inserire in s2 per fare in modo che il pacchetto di risposta sia riconosciuto come risposta al pacchetto inviato all'inizio dell'esercizio?

Soluzione:

```
priority = 20
match = [ eth_type=IPv4, ip_src=IP5, ip_proto=UDP ]
action=[ SetField(ipv4_src=IP3), SetField(eth_src=MAC3), output(FLOOD) ]
```

Tabella 1: Regole iniziali

Switch s1		
Priority	Match	Action
5	*	Output(FLOOD)
10	eth_type = IPv4 ip_proto=UDP	Output(CONTROLLER)
15	eth_dst = MAC2	DecTTL, Output(2)
15	eth_dst = MAC1	DecTTL, Output(1)

Switch s2		
Priority	Match	Action
0	*	Output(FLOOD)
10	eth_type = IPv4 ip_proto = TCP	Output(CONTROLLER)
10	eth_type = IPv4 ip_proto = UDP	Output(CONTROLLER)

Switch s3		
Priority	Match	Action
0	*	Output(FLOOD)
15	eth_dst = MAC5	DecTTL, Output(2)
15	eth_dst = MAC6	DecTTL, Output(3)

Programma 1 – Packet In

```
@set_ev_cls(ofp_event.EventOFPPacketIn, MAIN_DISPATCHER)
def packet_in_handler(self, ev):
   msg = ev.msg; datapath = msg.datapath; ofproto = datapath.ofproto;
   parser = datapath.ofproto_parser; in_port = msg.match['in_port']
   pkt = packet.Packet(msg.data); pkt_eth = pkt.get_protocol(eth.eth)
   pkt_ipv4 = pkt.get_protocol(ipv4.ipv4)
   pkt_udp = pkt.get_protocol(udp.udp); pkt_tcp = pkt.get_protocol(tcp.tcp)
   data = msg.data

   if (datapath.id == 2 and pkt_udp is not None and pkt_ipv4.dst == IP3):
      match = parser.OFPMatch(eth_type=ETH_TYPE_IP,
         ipv4_dst = pkt_ipv4.dst, ip_proto=IPPROTO_UDP,
         udp_dst=pkt_udp.dst_port)
      actions = [
         parser.SetField(eth_dst = MAC5),
         parser.SetField(ipv4_dst = IP5),
         parser.OFPActionOutput(3) ]
      inst = [ parser.OFPInstructionActions(
         ofproto.OFPIT_APPLY_ACTIONS,actions)]
      mod = parser.OFPFlowMod(datapath=datapath, priority=20,
         match=match, instructions=inst)
      datapath.send_msg(mod)
      return

   actions = [parser.OFPActionOutput(ofproto.OFPP_FLOOD)]
   out = parser.OFPPacketOut(datapath=datapath,
            buffer_id=ofproto.OFP_NO_BUFFER,data=msg.data,
            in_port=in_port, actions=actions)
   datapath.send_msg(out)
```

Esercizio 2

Si consideri un'interfaccia di uscita gestita da uno scheduler Deficit Round Robin che trasmette un byte per nanosecondo. Al tempo $t = 0$ si presentano al sistema i seguenti pacchetti, ordinati dal più recente al più vecchio la cui lunghezza è indicata in byte:

Coda 1 150,75,125

Coda 2 50,150,225

Coda 3 100,50,175

I quanti sono tutti di 200 byte. Al tempo $t = 350\,\text{ns}$ si presenta un pacchetto da 30 byte alla coda 1. È inoltre presente un meccanismo di gestione del buffer che scarta tutti i pacchetti che, quando viene il loro turno, non cominciano la trasmissione entro $T = 600\,\text{ns}$ dal loro arrivo.

7. Quali pacchetti vengono trasmessi primo round?

Soluzione:

Round	Coda	Deficit iniziale	Deficit finale	Tempo di trasmissione	Tempo di fine trasmissione
1	1		75	125	125
		75	0	75	200
	2		200		
	3		25	175	375
2	1	0	50	150	525
		0	0	30	555
	2	200	175	225	780
		175	175	150 scartato	
		175	0	50 scartato	
	3	25		50 scartato	
				100 scartato	

Al primo round vengono trasmessi i seguenti pacchetti. Coda 1: 125, 75. Coda 3: 175

8. Qual è il primo pacchetto scartato?

Soluzione: Il secondo pacchetto della coda 2

9. Quando inizia la trasmissione l'ultimo pacchetto della coda 1?

Soluzione: Al tempo $t = 525\,\text{ns}$

Si assuma ora che i quanti siano rispettivamente di 150, 225 e 175 byte per le code 1, 2 e 3.

10. Supponendo che le code siano infinitamente piene, qual è il throughput della coda 1?

Soluzione:

$$\frac{150}{150+225+175}\frac{8\,\text{bit}}{10^{-9}\,\text{s}} = 2{,}18\,\text{Gbit/s}$$

Esercizio 3

Un sistema di controllo degli accessi conserva una black list di indirizzi MAC cui è vietato l'accesso in rete. La black list usa una *cuckoo* hash table composta da due array di 400 elementi ciascuno. Si consideri una coppia di funzioni hash h_1 e h_2 ideali usate per indicizzare gli array.

Si consideri di aver effettuato l'inserimento di 400 indirizzi MAC.

11. Quali sono le lunghezze (in bit) degli output delle funzioni hash?

Soluzione: Entrambe le funzioni hanno un output di $\lceil \log_2 400 \rceil = 9$ bit.

12. Calcolare il numero medio di indirizzi MAC inseriti in ogni array.

Soluzione: Ci aspettiamo che ciascun array contenga $400/2 = 200$ elementi.

13. Calcolare la probabilità, nel caso medio, che il prossimo elemento inserito espella dal primo array un elemento già presente?

Soluzione: La probabilità è $1/2$.

14. Calcolare la probabilità di falso positivo.

Soluzione: Le cuckoo hash table non danno falsi positivi.

15. Dare una stima dell'occupazione di memoria della tabella.

Soluzione:

Ciascun array contiene un indirizzo MAC da 48 bit, per un totale di:

$$2 \times 400 \times 48 = 38\,400\,\text{bit}$$

16. Si supponga che l'indirizzo x sia già inserito nel primo array. Deve essere inserito nel primo array anche il nuovo elemento y, tale per cui $h_1(y) = h_1(x)$. Cosa viene inserito nel secondo array e dove?

Soluzione: L'elemento x viene spostato nel secondo array nella posizione $h_2(x)$.